
ИММЕРСИВНЫЙ / LEAD MAGNET

Fable 5 Prompt Guide

Рабочий мини-гайд по постановке задач сильным Claude-моделям: Fable 5, Opus 4.8 и Sonnet 5.

Audience	Claude Code, AI systems, video production, sites, prototypes
Inside	Rules, templates, checklist, where an expensive model pays off
Channel	https://t.me/+afIVRcluSel2OTYy
Handle	@Art_Cog

Что это

Это не магическая коллекция "секретных промптов". Это короткая рабочая инструкция: как давать дорогой и сильной модели задачу так, чтобы она не тратила лимит на лишнее, не расползалась в абстракции и быстрее доходила до результата.

Главная идея простая: чем сильнее модель, тем важнее не писать ей больше текста, а точнее задавать рамку работы.

1. Когда брать Fable 5

Используйте Fable 5 не для рутины, а для задач, где ошибка дорого стоит или где нужно удержать много контекста.

Подходит:

- архитектура проекта и план реализации;
- ревью сложного решения;
- поиск неочевидных ошибок;
- дебаг, где проблема может быть не только в коде, но и в логике продукта;
- финальная проверка важного промпта, гайда, лендинга, сценария или инструмента;
- создание первого сильного прототипа, если качество важнее экономии.

Не обязательно брать Fable 5:

- простая правка текста;
- маленький CSS-баг;
- переименование файлов;
- генерация однотипных блоков;
- задача, где хороший результат даст более дешевая модель.

Правило для себя: если задача не требует глубины, не сжигайте дорогую модель.

2. Перед задачей дайте контекст

Сильная модель лучше работает, когда понимает, зачем вы это делаете.

Плохой запрос:

Сделай сайт.

Рабочий запрос:

Я готовлю видео для русскоязычного YouTube-канала про AI-инструменты. Мне нужен демонстрационный сайт, который зритель увидит в сравнении трех моделей. Важно: русский язык, быстро открывается в браузере, визуально понятно, есть рабочие интерактивные элементы. Создай результат так, чтобы его можно было сразу показать на записи.

Контекст не должен быть длинным. Достаточно объяснить:

- для кого результат;
- где он будет использоваться;
- что считается успехом;
- чего делать не нужно.

3. Давайте критерии успеха

Модель не обязана угадать, что для вас "хорошо". Скажите, как вы будете оценивать результат.

Мини-шаблон:

Сделай [результат].

Критерии:

1. Должно открываться локально без сложной установки.
2. Весь текст на русском.
3. Главный сценарий должен работать.
4. Визуально должно быть понятно, что это за продукт.
5. В конце коротко напиши, что сделано, как открыть и что проверить.

Это особенно важно для тестов моделей: без критериев вы сравниваете не модели, а случайность.

4. Ограничивайте лишнюю инициативу

Сильные модели часто стараются "помочь шире", чем нужно: добавляют архитектуру, фичи, обертки, тестовые слои и будущие расширения.

Если вам нужен быстрый результат, скажите это прямо:

Не расширяй задачу сверх запроса.

Не добавляй функции "на будущее".

Если видишь несколько вариантов, выбери один практичный и реализуй его.

После результата дай короткий отчет: что сделано, как открыть, что осталось спорным.

Это не делает модель слабее. Это направляет ее силу в нужную точку.

5. Для кода просите проверку результата

Если модель пишет приложение, сайт или игру, попросите не только создать файлы, но и проверить главный сценарий.

Шаблон:

После реализации проверь результат как пользователь.
Если это HTML-страница - открой ее или объясни, как открыть.
Если это игра - проверь управление, проигрыш, победу и перезапуск.
Если это инструмент - проверь основной сценарий от входных данных до результата.
В отчете отдельно укажи: что проверено руками или автотестом, а что не проверено.

Слабый отчет: "готово".

Полезный отчет: "открыл, нажал, проверил, нашел ограничение".

6. Для сложных задач просите разбор, а не уверенность

На сложных задачах опаснее всего не ошибка, а уверенный тон без доказательств.

Хорошая формулировка:

Если ты не уверен, не делай вид, что проверил.
Отделяй факты от предположений.
Если нашел проблему, покажи, по каким признакам ты сделал вывод.
Если есть спорный момент, вынеси его в отдельный блок.

Это особенно полезно для:

- анализа YouTube-метрик;
 - ремонта чужого проекта;
 - проверки логики продукта;
 - сравнения моделей;
 - поиска скрытых дефектов.
-

7. Как использовать CLAUDE.md и память

Если вы регулярно работаете над одним проектом, не держите все правила в голове и не повторяйте их в каждом чате.

В Claude Code можно хранить проектные инструкции в `CLAUDE.md`. Туда стоит класть только то, что должно повторяться в каждой сессии:

- структура проекта;
-

- команды запуска и тестов;
- стиль кода;
- что нельзя трогать;
- как оформлять результат;
- где лежат важные файлы.

Не превращайте `CLAUDE.md` в книгу. Короткие правила обычно работают лучше длинной стены текста.

8. Где дорогая модель реально окупается

По итогам тестов для видео "Fable 5 vs Opus 4.8 vs Sonnet 5" самый важный вывод такой:

Дорогая модель оправдывает цену не всегда. Она особенно сильна там, где нужно увидеть не только видимую задачу, но и скрытую логику.

Примеры:

- не просто сверстать сайт, а сделать его цельным и убедительным;
- не просто починить код, а понять, почему продукт советует неправильное решение;
- не просто построить график, а объяснить, что из него следует;
- не просто выполнить промпт, а оставить проверяемый отчет.

Если задача простая, дешевле взять более легкую модель. Если задача комплексная, экономия на модели может обернуться потерей времени.

9. Универсальный промпт для сильной модели

Ты работаешь над важной задачей, где качество результата важнее скорости.

Контекст:

[коротко объясни, зачем нужен результат и где он будет использоваться]

Задача:

[что нужно сделать]

Критерии успеха:

1. [главный критерий]
2. [что должно работать]
3. [что должно быть понятно пользователю]
4. [ограничение по стилю, языку, формату]

Ограничения:

- Не добавляй лишние функции сверх задачи.
- Не делай вид, что проверил то, что не проверял.
- Если есть спорные решения, вынеси их отдельно.

Финальный отчет:

1. Что сделано.
2. Как открыть или проверить.
3. Что проверено.
4. Что осталось ограничением.

10. Мини-чеклист перед отправкой

Перед тем как нажать Enter, проверьте:

- понятно ли, зачем делается задача;
- указано ли, что считать хорошим результатом;
- есть ли ограничение "не делать лишнее";
- сказано ли, как отчитываться;
- указано ли, где сохранить результат;
- есть ли требование проверить главный сценарий.

Если на эти вопросы есть ответы, промпт уже сильнее большинства случайных запросов.

Источники и проверка

Материал пересобран под "ИИмерсивный" как оригинальный рабочий гайд. Исходный PDF использовался только как референс по типу материала, без переноса чужого текста, оформления, ссылок и авторской подачи.

Сверено с официальными материалами Anthropic:

- Prompting best practices: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>
- Prompt engineering overview: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
- Claude Code memory / CLAUDE.md: <https://code.claude.com/docs/en/memory>
- Context engineering for AI agents: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Больше файлов, промптов и разборов: <https://t.me/+afIVRcluSel2OTYy>